

Pràctiques EI1027 - Sessió 6

Aplicacions Web: Gestió de sessions i autenticació d'usuaris

1. Autenticació d'usuaris i gestió de sessions

En la primera part de la sessió treballarem una tècnica fonamental en el desenvolupament de sistemes d'informació basats en web: el control de sessions, i en particular la seua aplicació a l'autenticació d'usuaris.

1.1 Gestió de sessions

En el protocol HTTP, en què es basa la Web, cada petició al servidor és independent de la resta: no es manté cap informació (o "estat") entre peticions. En canvi, en el desenvolupament d'aplicacions sí és necessari mantindre l'estat: per exemple, després que un usuari s'autentique en el sistema volem mantindre eixa informació, i no fer que l'usuari haja de tornar a posar l'usuari i contrasenya contínuament per a fer qualsevol acció.

Anomenem *sessió* al conjunt d'informació que volem mantindre entre peticions. El cicle de vida d'una sessió en una aplicació basada en servlets (com és, també, Spring MVC) és el següent:

1. Una sessió s'inicia quan el servidor rep una nova connexió des d'un navegador des del que no hi havia cap sessió oberta. El servidor assigna automàticament un identificador únic a la nova sessió.
2. La nova sessió pot contindre qualsevol objecte Java, que es guardarà mentres dure la sessió. Aquests objectes es guarden com a *atributs* de la sessió, donant-los un nom únic.
3. A partir d'ací, la sessió es mantindrà fins que passe una d'aquestes coses:
 - a. La sessió estiga inactiva (és a dir, sense cap petició al servidor) durant més d'un temps predeterminat. En el servidor web que usem en les pràctiques (Tomcat), per defecte aquest *timeout* és de 30 minuts, però com veurem es pot configurar un valor per a cada aplicació.
 - b. La sessió s'invalida explícitament.

En ambdós casos, tots els atributs de la sessió s'esborraran automàticament i passaran a valdre null.

Les dades de la sessió estan encapsulades en la classe [HttpSession](#), que té els següents mètodes d'especial interès:

- `String getId()` Torna l'identificador únic de la sessió.
- `void invalidate()` Invalida la sessió immediatament, i elimina de la sessió tots els objectes que s'hi hagueren afegit.

- `long getCreationTime()` Torna la marca temporal en què es va crear la sessió (en mil·lisegons des de la mitjanit de l'1 de gener de 1970 GMT (fàcilment processable amb les classes del paquet `java.time`)).
- `long getLastAccessedTime()` Torna la marca temporal de l'última petició al servidor associada a aquesta sessió (mateix format que `getCreationTime()`).
- `boolean isNew()` Torna `True` si és una sessió nova que s'està creant. Útil per a reenviar a l'usuari a una pàgina de benvinguda, i també per exigir autenticació immediatament.
- `void removeAttribute(String name)` Elimina de la sessió l'objecte indicat.
- `void setAttribute(String name, Object value)` Afegeix un objecte a la sessió amb el nom especificat.
- `Object getAttribute(String name)` Torna l'objecte de la sessió amb el nom especificat, o `null` si no existeix.
- `int getMaxInactiveInterval()` Torna el màxim interval sense peticions, en segons, que s'esperarà abans d'invalidar automàticament la sessió.
- `void setMaxInactiveInterval(int interval)` Especifica el temps màxim d'inactivitat permès abans d'invalidar automàticament la sessió.
- `Enumeration getAttributeNames()` Torna els noms de tots els objectes que s'han afegit a la sessió.

Aquest objecte és independent del mètode que s'utilitzi per a passar la informació de la sessió entre client i servidor. Tenim almenys aquestes opcions:

- Elements HTML `<hidden>` per a passar l'identificador de sessió. És poc flexible, perquè només es pot utilitzar dins de formularis.
- Reescriptura de URLs: el servidor afegeix automàticament un paràmetre a cada pàgina servida al client amb l'identificador de sessió.
- *Cookies*: Són fitxers que guarda localment el navegador i que poden mantindre informació addicional, a més de l'identificador de sessió. A més són persistents (es mantenen inclús si es tanca el navegador o s'apaga l'ordinador), cosa que els fa imprescindibles en moltes aplicacions.

En Spring tots tres mètodes estan implementats, i es pot utilitzar el mètode desitjat injectant la classe adient en la configuració de la vostra aplicació. Per defecte es crea automàticament una *cookie* quan és necessari.

Accedir a la informació de la sessió és molt fàcil, perquè cada petició a un servlet (`HttpRequest`) s'associa a un objecte `HttpSession`. Spring proporciona algunes possibilitats addicionals que faciliten la gestió de sessions en aplicacions grans, fent innecessari l'ús explícit de `HttpSession`:

- La definició d'objectes amb l'anotació `@SessionAttribute`, que els insereix automàticament en `HttpSession`.
- La definició de controladors com a d'àmbit de sessió, cosa que fa que tot el seu estat intern es mantinga automàticament com a part de la sessió¹.

¹ Els *àmbits* (*scopes*) són molt importants en les aplicacions web, i defineixen el cicle de vida dels objectes que creem. Normalment hi ha quatre àmbits: L'*àmbit de context* o *d'aplicació* conté objectes associats a l'aplicació, l'*àmbit de sessió* conté objectes associats a la sessió, l'*àmbit de petició* a cada petició *HTTP* i l'*àmbit de pàgina* a cada pàgina web.

Nosaltres no utilitzarem aquestes possibilitats, ja que per al nostre cas l'ús directe de `HttpSession` és molt senzill, i semblant a la manera de funcionar en altres *frameworks* de programació d'aplicacions web. En tot cas, pots consultar la documentació de Spring per si t'interessa utilitzar-les en el teu projecte (en molts casos et simplificaran el codi).

1.2 Autenticació d'usuaris

En molts casos la creació d'una nova sessió du associat un procés d'autenticació d'usuari. Cal que tingues en compte que això no és sempre així: autenticació i gestió de sessions són tècniques diferents i independents, i s'aplicaran juntes o no segons les necessitats particulars de cada aplicació.

Hi ha distints patrons d'autenticació d'usuaris en aplicacions Web. Nosaltres introduïrem ací un dels més utilitzats: associar els usuaris autenticats a una sessió. Així, una vegada es verifiqui la identitat de l'usuari dins d'una sessió, hi afegirem la informació de l'usuari com a un atribut de l'objecte `HttpSession`. D'aquesta manera, la sessió estarà associada a l'usuari fins que s'invalidei (per un *timeout* o perquè l'usuari decidisca tancar la sessió).

El primer pas deurà ser, en tot cas, crear una classe que represente les dades necessàries per a la identificació de l'usuari:

Llistat 1: `es.uji.ei1027.clubesportiu.model.UserDetails`

```
package es.uji.ei1027.clubesportiu.model;

public class UserDetails {
    String username;
    String password;

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }
}
```

Aquest és el model d'usuari més senzill possible: només hi mantenim un usuari i una contrasenya. En la realitat ací sol haver-hi molta més informació, tant de caire personal de l'usuari com per a la

gestió del sistema. En particular, típicament s'afegeix també una llista de permisos, o de rols que l'usuari està autoritzat a tindre; per exemple, en el nostre cas un usuari pot ser un usuari normal, o un administrador del sistema, amb distints nivells d'accés que cal gestionar.

Normalment aquestes dades residiran en una base de dades, però hi ha altres possibilitats (per exemple un servidor dedicat d'autenticació, com en la UJI). Per a abstraure aquest detall, crearem un DAO:

Llistat 2: es.uji.ei1027.clubesportiu.dao.UserDao

```
package es.uji.ei1027.clubesportiu.dao;

import java.util.Collection;
import es.uji.ei1027.clubesportiu.model.UserDetails;

public interface UserDao {
    UserDetails loadUserByUsername(String username, String password);
    Collection<UserDetails> listAllUsers();
}
```

El DAO només dona dos possibilitats: intentar autenticar un usuari amb nom i contrasenya, tornant-ne les dades si l'autorització és correcta; i llistar tots els usuaris, cosa que és una informació sensible i deu estar restringida a usuaris autoritzats, com després veurem.

En aquest punt podríem crear ja la corresponent taula Usuaris en la base de dades, però ho deixem com a exercici; ací simplement crearem una llista d'usuaris en memòria, només per a proves.

Encara que és un exemple, hem incorporat una llibreria real d'encryptació, i ens assegurem que les contrasenyes reals mai estiguen emmagatzemades en obert en l'aplicació. La llibreria es diu [Jasypt](#) (*Java Simplified Encryption*), i conté implementacions fàcils d'usar dels mecanismes d'encryptació de Java (que són molt potents, però un tant enrevesats). Recomanem llegir-ne la documentació, és molt informativa sobre seguretat bàsica en aplicacions Java.

La llibreria ja està incorporada al projecte (en altre cas, pots seguir els passos indicats [en aquest document](#)). Una vegada Jasypt està disponible, podem usar-la en la nostra implementació de prova d'un DAO per a carregar informació per a l'autenticació d'usuaris:

Llistat 3: es.uji.ei1027.clubesportiu.dao.FakeUserProvider.java

```
package es.uji.ei1027.clubesportiu.dao;

import java.util.Collection;
import java.util.HashMap;
import java.util.Map;
import org.jasypt.util.password.BasicPasswordEncryptor;
import org.springframework.stereotype.Repository;
import es.uji.ei1027.clubesportiu.model.UserDetails;

@Repository
public class FakeUserProvider implements UserDao {
```

```

        final Map<String, UserDetails> knownUsers = new HashMap<String,
UserDetails>();

    public FakeUserProvider() {
        BasicPasswordEncryptor passwordEncryptor =
            new BasicPasswordEncryptor();

        UserDetails userAlice = new UserDetails();
        userAlice.setUsername("alice");
        userAlice.setPassword(passwordEncryptor.encryptPassword("alice"));
        knownUsers.put("alice", userAlice);

        UserDetails userBob = new UserDetails();
        userBob.setUsername("bob");
        userBob.setPassword(passwordEncryptor.encryptPassword("bob"));
        knownUsers.put("bob", userBob);
    }

    @Override
    public UserDetails loadUserByUsername(String username, String password) {
        UserDetails user = knownUsers.get(username.trim());
        if (user == null)
            return null; // Usuari no trobat
        // Contrasenya
        BasicPasswordEncryptor passwordEncryptor =
            new BasicPasswordEncryptor();

        if (passwordEncryptor.checkPassword(password, user.getPassword())) {
            // Per seguretat el camp password no s'ha de tornar
            UserDetails safeUser = new UserDetails();
            safeUser.setUsername(user.getUsername());
            return safeUser;
        }
        else {
            return null; // bad login!
        }
    }

    @Override
    public Collection<UserDetails> listAllUsers() {
        return knownUsers.values();
    }
}

```

Com pots veure, en el constructor creem dos usuaris, “alice” (amb contrasenya “alice”) i “bob” (amb contrasenya “bob”).

Una vegada tenim el DAO implementat, necessitem un formulari i un controlador per a fer el *login* i obrir una sessió autenticada. El formulari és molt senzill. Crea el següent fitxer a l’arrel de la carpeta de les plantilles:

Llistat 4: login.html

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org" lang="ca">
<head>

```

```

<link rel="stylesheet" type="text/css"
      href="/css/natacio.css"
      th:href="@{/css/natacio.css}"/>
<title>Login</title>
<meta charset="UTF-8"/>
<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
rel="stylesheet" crossorigin="anonymous"/>
</head>
<body>
<main class="container">
  <h1>Accés</h1>
  <form action="#" th:action="@{/login}" th:object="${user}" method="post">

    <div class="row mb-3">
      <label for="username"
        class="col-sm-2 col-form-label">Nom d'usuari:</label>
      <div class="col-sm-3">
        <input id="username" class="form-control"
          type="text" th:field="*{username}" />
      </div>
      <div class="col-sm-4">
        <div th:if="${#fields.hasErrors('username')}"
          th:errors="*{username}" class="error">
        </div>
      </div>
    </div>

    <div class="row mb-3">
      <label for="password"
        class="col-sm-2 col-form-label">Contrasenya:</label>
      <div class="col-sm-3">
        <input id="password" class="form-control"
          type="password" th:field="*{password}" />
      </div>
      <div class="col-sm-4">
        <div th:if="${#fields.hasErrors('password')}"
          th:errors="*{password}" class="error">
        </div>
      </div>
    </div>

    <div class="row">
      <div class="col-sm-3 offset-sm-2">
        <button type="submit"
          class="btn btn-primary">Accedir</button>
      </div>
    </div>
  </form>
</main>
</body>
</html>

```

Com a únics comentaris, fixa't que en aquest cas l'acció del formulari definida en l'etiqueta <form> és un controlador dedicat que farà l'autenticació (/login), en lloc de ser el mateix controlador que

ha generat la vista, com hem fet fins ara.

El controlador el podem definir de la següent manera:

Llistat 5: es.uji.ei1027.clubesportiu.controller.LoginController.java

```
package es.uji.ei1027.clubesportiu.controller;

import jakarta.servlet.http.HttpSession;
import es.uji.ei1027.clubesportiu.dao.UserDao;
import es.uji.ei1027.clubesportiu.model.UserDetails;

import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.stereotype.Controller;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.validation.BindingResult;
import org.springframework.ui.Model;

@Controller
public class LoginController {

    @Autowired
    private UserDao userDao;

    @RequestMapping("/login")
    public String login(Model model) {
        model.addAttribute("user", new UserDetails());
        return "login";
    }

    @RequestMapping(value="/login", method=RequestMethod.POST)
    public String checkLogin(@ModelAttribute("user") UserDetails user,
                             BindingResult bindingResult, HttpSession session) {
        UserValidator userValidator = new UserValidator(); ①
        userValidator.validate(user, bindingResult);
        if (bindingResult.hasErrors()) {
            return "login";
        }
        // Comprova que el login siga correcte
        // intentant carregar les dades de l'usuari
        user = userDao.loadUserByUsername(user.getUsername(),
                                           user.getPassword()); ②

        if (user == null) {
            bindingResult.rejectValue("password", "badpw",
                                      "Contrasenya incorrecta");

            return "login";
        }
        // Autenticats correctament.
        // Guardem les dades de l'usuari autenticat a la sessió ③
        session.setAttribute("user", user);
        // Ací podem restaurar on tornar si ho existeix en la sessió, ④
        // i en altre cas...
        // Torna a la pàgina principal
    }
}
```

```

        return "redirect:/";
    }

    @RequestMapping("/logout")
    public String logout(HttpSession session) {
        session.invalidate();
        return "redirect:/";
    }
}

```

Hi ha una primera validació del resultat del formulari ① i després la comprovació de la existència de l'usuari i la correcció de la contrasenya ②. Si hi ha algun error torna a login. Si no hi ha errors, s'afegeix la informació de l'usuari a la sessió ③. Ací caldria comprovar quina és l'adreça on volem anar a continuació ④ (com veurem a continuació) i, si no n'hi ha cap, tornem a la pàgina d'inici.

El controlador del login necessita d'un validador de les dades del formulari, que serà com aquest:

Llistat 6: es.uji.ei1027.clubesportiu.controller.UserValidator.java

```

package es.uji.ei1027.clubesportiu.controller;

import org.springframework.validation.Errors;
import org.springframework.validation.Validator;

import es.uji.ei1027.clubesportiu.model.UserDetails;

class UserValidator implements Validator {

    @Override
    public boolean supports(Class<?> cls) {
        return UserDetails.class.isAssignableFrom(cls);
    }

    @Override
    public void validate(Object obj, Errors errors) {
        // Exercici: Afegeix codi per comprovar que
        // l'usuari i la contrasenya no estiguen buits
        // ...
    }
}

```

Exercici 1:

- Completa el mètode `validate` d'aquest validador.
- Executa el login amb `http://localhost:8080/login` i comprova que la validació funcione bé.

A continuació, crearem una zona privada per accedir sols mitjançant l'autenticació.

1.3 Una zona privada a la web

Anem a utilitzar tot aquest treball per crear una “zona privada” a la web. Aquesta zona permetrà consultar la llista d'usuaris coneguts pel sistema. El controlador és també molt senzill:

Llistat 7: UserController.java

```
package es.uji.ei1027.clubesportiu.controller;

import jakarta.servlet.http.HttpSession;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.RequestMapping;

import es.uji.ei1027.clubesportiu.dao.UserDao;
import es.uji.ei1027.clubesportiu.model.UserDetails;

@Controller
@RequestMapping("/user")
public class UserController {
    private UserDao userDao;

    @Autowired
    public void setSociDao(UserDao userDao) {
        this.userDao = userDao;
    }

    @RequestMapping("/list")
    public String listSocis(HttpSession session, Model model) {
        if (session.getAttribute("user") == null) {
            model.addAttribute("user", new UserDetails());
            return "login"; ①
        }
        model.addAttribute("users", userDao.listAllUsers());
        return "user/list";
    }
}
```

Fixa't que, com a únic punt a destacar, si la sessió no conté un usuari autenticat s'invoca la vista *login*, perquè l'usuari s'autentique ①.

El llistat és igualment senzill:

Llistat 8: user/list.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org" lang="ca">
<head>
    <title>Llista d'usuaris del sistema</title>

    <meta charset="UTF-8"/>
```

```

<link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
crossorigin="anonymous"/>
<link rel="stylesheet" type="text/css" href="/css/natacio.css"
th:href="@{/css/natacio.css}"/>
</head>

<body>
<main class="container">
  <h1>Llista d'usuaris del sistema</h1>

  <div th:if="${#lists.isEmpty(users)}">
    No hi ha usuaris
  </div>

  <table th:unless="${#lists.isEmpty(users)}"
    class="table table-condensed">
    <thead>
      <tr>
        <th>Nom d'usuari</th>
      </tr>
    </thead>

    <tbody>
      <tr th:each="user: ${users}">
        <td th:text="${user.username}"></td>
      </tr>
    </tbody>
  </table>

  <a href="/" class="btn btn-primary" role="button">Tornar</a>
</main>

</body>
</html>

```

Exercici 2:

- En l'arxiu *application.properties* afegiu la següent declaració, que canvia el *timeout* de les sessions a 1 minut. Així facilitarem les proves:

```
server.session.timeout=1
```

- Comprova que al accedir a la pàgina *user/list* la primera volta, demana l'autenticació. Per tal de fer-ho recorda modificar *index.html* per afegir l'enllaç a la llista d'usuaris. També podem afegir *logout*.

Exercici 3:

Que el controlador *LoginController* torne a la pàgina principal després de l'autenticació no és gens lògic. És molt més útil que, una vegada autenticat l'usuari, simplement es redirigisca a la pàgina a què l'usuari volia anar. Per a això necessitem mantindre la URL de la pàgina, la qual

podem guardar en un atribut de la sessió; recorda que la sessió és semblant a un Map, i podem afegir-hi tots els atributs que vulguem. Implementa-ho seguint aquests passos:

- En el mètode `UserController.listSocis`, quan l'usuari no està autenticat i cal fer `/login`, afegeix un nou atribut a la sessió anomenat per exemple `nextUrl`, que continga l'URL a què l'usuari estava intentant accedir (en aquest cas, `/user/list`).
- Modifica el codi d'autenticació (`LoginController.checkLogin`) per a comprovar si hi ha definit un atribut `nextUrl` a la sessió, i en eixe cas que faci la redirecció a eixa URL en compte de la pàgina principal. Compte, per a evitar confusions recorda eliminar l'atribut de la sessió una vegada l'hages utilitzat!
- ¿Deus utilitzar “`redirect:`” amb aquesta `nextUrl`? ¿Per què?
- Comprova de nou que, efectivament, la validació s'està realitzant correctament, i només pots accedir a la pàgina `user/list` si estàs autenticat, i que ara `LoginController` torna a `/user/list` i no a `index`.

2. Formatat de pàgines amb Thymeleaf

2.1 Dissenys globals per a l'aplicació web

L'últim punt que anem a tractar és el disseny global de les pàgines que composen l'interfície d'usuari de la vostra aplicació web. Com sabeu, és molt important que el disseny siga consistent, cosa que implica reutilitzar els mateixos elements de disseny: la capçalera de cada pàgina, el menú de navegació, etcètera.

No és una bona pràctica copiar el codi HTML dels elements en cada pàgina: idealment, caldria definir els elements una única vegada i després reutilitzar-los, de tal manera que baste modificar-los una vegada perquè els canvis s'apliquen automàticament a tota la web.

Thymeleaf (o qualsevol altra tecnologia de vistes que utilitzeu) proporciona capacitats excel·lents per a aconseguir aquest objectiu. La ferramenta principal que ofereix són els *fragments*, que permeten definir segments de codi HTML que es poden personalitzar i incloure en qualsevol vista. Podeu consultar la [documentació de Thymeleaf](#) per a trobar tots els detalls.

Thymeleaf és molt flexible a l'hora de definir els fragments: Es pot definir cada fragment en un arxiu diferent; també poden agrupar-se distints fragments en un sol fitxer; i fins i tot és possible carregar fragments definits en un lloc extern. Per simplicitat, en esta pràctica definirem cada fragment en un arxiu diferent, i els agruparem tots en una nova carpeta `fragments` a la carpeta de les vistes (`src/main/resources/templates`).

2.2 Un fragment per indicar l'usuari autenticat

En la interfície d'usuari tal i com la tenim ara no s'indica si l'usuari està connectat o no, cosa que resulta molt confusa. Anem a escriure un fragment anomenat `logininfo` que mostre aquesta informació.

Llistat 9: `src/main/resources/templates/fragments/logininfo.html`

```

<div class="loggeduser" th:object="${session}" th:fragment="logininfo"> ①
  <p th:if="*{user == null}"> ②
    No autenticat <a href="/login" th:href="@{/login}">Entrar</a> ③
  </p>
  <p th:unless="*{user == null}">
    Autenticat com <span th:text="*{user.username}"></span>
    <a href="/logout" th:href="@{/logout}">Eixir</a>
  </p>
</div>

```

En primer lloc, com a dades del fragment usem `session` ①, que està disponible automàticament en les vistes i permet accedir a la informació de sessió (és equivalent a usar `getSession()` en Java). Després, simplement usem, `th:if` i `th:unless` per a mostrar un codi HTML o un altre segons si hi ha un usuari autenticat o no.

És important que en fragments utilitzes adreces relatives ③ per a evitar els problemes que poden derivar-se de dependre de la URL arrel (en producció, una web pot estar penjada a una URL que no és la de desenvolupament). Amb `th:href="@{/login}"`, començant amb `"/"`, estem indicant una adreça relativa al context. Per exemple, si la nostra aplicació està instal·lada a `http://localhost:8080/myapp`, la URL `th:href="@{/login}"` es reescriuria com a `href="/myapp/login"`. Les adreces absolutes tan sols deurien utilitzar-se quan el recurs resideix en un servidor diferent. [Ací hi ha més informació sobre com indicar URLs](#) relatives al context, relatives al servidor, etc.

Per a completar el treball, afegeix la següent classe al teu fitxer CSS:

```

.loggeduser {
  color: #343a40;
  text-align: right;
  margin-right: 10pt;
}

```

I, a partir d'aquest moment, podem importar i utilitzar el nou fragment `logininfo` en qualsevol vista. Per exemple, a `user/list.html` podem afegir aquest fragment de la següent manera:

```

...
<h1>Llista d'usuaris del sistema</h1>
<div th:replace="~{fragments/logininfo :: logininfo}" >...</div>
...

```

El text dins del `div` és irrellevant, perquè serà reemplaçat per el fragment.

El resultat en el cas de la vista `user/list` es mostra en la Figura 1.

Exercici 4:

- Crea un nou fragment, de nom `menu`, en un fitxer `menu.html`, amb aquesta estructura:

```

<nav th:fragment="menu">
  ...
</nav>

```

que genere la llista següent, usant les etiquetes de llista `ul` i `li`:

- Pàgina principal
- Proves i classificacions (enllaç a /prova/list)
- Àrea privada (enllaç a /user/list)
- Cada element deu ser un enllaç a la pàgina corresponent. Ves amb compte: En general no és possible saber a priori en quina pàgina s'insereix una etiqueta personalitzada. Per tant, no és convenient usar URLs relatius en els enllaços. Per a fer els URLs relatius a l'arrel de la web, recorda usar `th:href="@{/...}"` com hem fet en `logininfo.html`.
- Afegeix el fragment menu davant del fragment logininfo.

Llista d'usuaris del sistema

Autenticat com bob [Eixir](#)

Nom d'usuari

bob

alice

Tornar

Figura 1. Efecte del fragment logininfo.

Exercici 5:

- Modifica la definició de *menu.html* per a adoptar els estils de barra de navegació de Bootstrap. Deu mostrar una barra de navegació, usant les etiquetes i classes de Bootstrap 5 relatives als Navbar ([enllaç](#)). Per exemple, el format del llistat 10 produeix el resultat de la Figura 2.

Llistat 10: menu.html amb una *navbar* de Bootstrap

```
<nav class="navbar navbar-expand-lg navbar-light bg-light" th:fragment="menu">
  <div class="navbar-collapse collapse">
    <ul class="navbar-nav mr-auto">
      <li class="nav-item">
        <a class="nav-link" href="/" th:href="@{/}">Pàgina principal</a>
      </li>
      ...
    </ul>
  </div>
</nav>
```

Llista d'usuaris del sistema

[Pàgina principal](#) [Proves i classificacions](#) [Àrea privada](#)

Autenticat com bob [Eixir](#)

Nom d'usuari

bob

alice

Tornar

Figura 2. Efecte del fragment menu, amb format de navbar de Bootstrap 5.

2.3 Disseny de pàgines completes

Recordem que el nostre objectiu final és obtenir dissenys de pàgines reutilitzables, i això es pot aconseguir en Thymeleaf definint una pàgina genèrica (coneguda com a *layout*) que defineix tots els elements comuns (capçalera, peu, ...), i en la qual s'inserta el contingut com un fragment.

La pàgina que farem servir com a base per a l'aplicació és la següent:

Llistat 11: fragments/base.html

```
<!DOCTYPE html>
<html lang="ca" xmlns:th="http://www.thymeleaf.org"
          xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout">
<head>
  <!-- Required meta tags -->
  <meta charset="UTF-8"/>
  <meta name="viewport" content="width=device-width, initial-scale=1,
shrink-to-fit=no"/>

  <!-- Bootstrap CSS -->
  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
crossorigin="anonymous"/>
  <!-- Bootstrap JS bundle para habilitar componentes interactivos como dropdowns
-->
  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
crossorigin="anonymous"></script>

  <!-- app css -->
  <link type="text/css" th:href="@{/css/natacio.css}" href="/css/natacio.css"
rel="stylesheet"/>

  <title layout:title-pattern="$CONTENT_TITLE">Club de natació</title>
</head>

<body>
<main class="container">
```

```

<header class="page-header">
  <h1>Campionat de Natació</h1>
</header>

<div th:replace=~{fragments/menu :: menu}></div>
<div th:replace=~{fragments/logininfo :: logininfo}></div>

<section layout:fragment="content">
  <!-- El contingut específic de cada pàgina es col·locarà aquí -->
</section>

<footer>
  <hr>
  <p class="text-body-secondary">EI1027 - Disseny i Implementació de Sistemes
d'Informació</p>
</footer>
</main>
</body>
</html>

```

Fixa't en els següents punts:

1. El disseny incorpora els fragments que ja hem definit: `nav` i `logininfo`.
2. El *Thymeleaf Layout Dialect* permet utilitzar noves etiquetes que porten el prefix `layout:`. Pots consultar-ne la [documentació de referència](#).
3. El punt més important és on s'inserirà el contingut al disseny, que està indicat per `layout:fragment="content"`
4. A més, també s'utilitza `layout:title-pattern`, que permet usar el mateix títol del fragment importat.

Això completa la definició del nou estil de pàgina. Aplicar-lo és molt senzill. Per exemple, anem a editar la vista de login d'aquesta manera:

Llistat 12: Nou login.html

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org" lang="ca"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
      layout:decorate=~{fragments/base}>
<head>
  <title>Login</title>
</head>
<body>
<section layout:fragment="content">
  <h2>Accés</h2>
  <form action="#" th:action="@{/login}" th:object="${user}" method="post">
    <div class="row mb-3">
      <label for="username"
        class="col-sm-2 col-form-label">Nom d'usuari:</label>
      <div class="col-sm-3">
        <input id="username" class="form-control"

```

```

        type="text" th:field="*{username}" />
    </div>
    <div class="col-sm-4">
        <div th:if="${#fields.hasErrors('username')}}"
            th:errors="*{username}" class="error">
        </div>
    </div>
</div>

<div class="row mb-3">
    <label for="password"
        class="col-sm-2 col-form-label">Contrasenya:</label>
    <div class="col-sm-3">
        <input id="password" class="form-control"
            type="password" th:field="*{password}" />
    </div>
    <div class="col-sm-4">
        <div th:if="${#fields.hasErrors('password')}}"
            th:errors="*{password}" class="error">
        </div>
    </div>
</div>

<div class="row">
    <div class="col-sm-3 offset-sm-2">
        <button type="submit"
            class="btn btn-primary">Accedir</button>
    </div>
</div>
</form>
</section>
</body>
</html>

```

Hem eliminat el codi que ja està inclòs a la pàgina base (head), excepte el títol que el volem per al "\$CONTENT_TITLE". Hem necessitat afegir només dos nous atributs, marcats en groc. La resta de la vista és completament igual:

1. En primer lloc, amb `layout:decorate` declarem que usarem la pàgina genèrica.
2. En segon lloc, amb `layout:fragment="content"` especifiquem quin element de la pàgina inserirem al lloc corresponent de la pàgina genèrica.

El resultat es mostra en la Figura 3.

Campionat de Natació

[Pàgina principal](#) [Proves i classificacions](#) [Àrea privada](#)

No autenticat [Entrar](#)

Accés

Nom d'usuari:

Contrasenya:

Accedir

El1027 - Disseny i Implementació de Sistemes d'Informació

Figura 3. Pàgina de login usant la pàgina genèrica.

2.4 Temes

Crear els estils per a una aplicació completa és molt costós. En general es parteix d'un *tema*, un conjunt d'estils ja creat per un dissenyador professional, i que només cal adaptar. Per a Bootstrap és possible trobar molts temes, tant d'ús gratuït com comercials. Per exemple, pots provar els de [Bootswatch](#), que només requereixen canviar l'enllaç al CSS de Bootstrap. Hi ha molts recursos similars que pots provar.

Campionat de Natació

[Pàgina principal](#) [Proves i classificacions](#) [Àrea privada](#)

No autenticat [Entrar](#)

Accés

Nom d'usuari:

Contrasenya:

Accedir

El1027 - Disseny i Implementació de Sistemes d'Informació

Figura 4. Exemple amb el tema de Bootswatch Cerulean.

3. Treball personal

- Aplica la pàgina genèrica que acabes de definir a la resta de pàgines de l'aplicació. Fixa't especialment en els estils que defineix Bootstrap per a taules i formularis.
- Assegura't amb WAVE que cada pàgina compleix amb bones pràctiques d'accessibilitat.

- Si vols adaptar també la pàgina principal, hauràs de convertir-la en una vista, servida per un controlador, perquè siga processada per Thymeleaf.
- Busca un estil de disseny de pàgina genèrica que s'adapte a les necessitats del teu projecte, i ves experimentant amb ell.

És important que experimentes amb les possibilitats de Bootstrap; et deixem una captura de pantalla com a guia:

Campionat de Natació

[Pàgina principal](#) [Gestió d'usuaris](#) [Àrea privada](#)

No autenticat. [Entrar](#)

Llista de Nadadors

Num. Federat	Nom	País	Edat	Gènere	Accions	
W12905567	Albert Puig	Espanya	21	Masculí	Edita	Esborra
Y76232424	Anastasia Davidova	Rússia	27	Femení	Edita	Esborra
131441141L	Aschwin Wildeboer	Espanya	33	Masculí	Edita	Esborra
XA1235690	Franziska van Almsick	Alemanya	30	Femení	Edita	Esborra
A123450987	Gemma Mengual	Espanya	33	Femení	Edita	Esborra
AP093456Q	Michael Phelps	USA	23	Masculí	Edita	Esborra
U65411124	Miguel Durán	Espanya	20	Masculí	Edita	Esborra
A012987523	Mireia Belmonte Garcia	Espanya	18	Femení	Edita	Esborra
J46644611	Natalia Ischenko	Rússia	28	Femení	Edita	Esborra
J4524611	Ona Carbonell	Espanya	30	Femení	Edita	Esborra
A4354664	Paola Tirados	Espanya	35	Femení	Edita	Esborra
U12777890	Ryan Lochte	USA	27	Masculí	Edita	Esborra
R466441788	Svetlana Romashina	Rússia	23	Femení	Edita	Esborra

[Afegeix nadador](#)

Campionat de Natació

[Pàgina principal](#) [Proves i classificacions](#) [Àrea privada](#)

No autenticat [Entrar](#)

Competicions

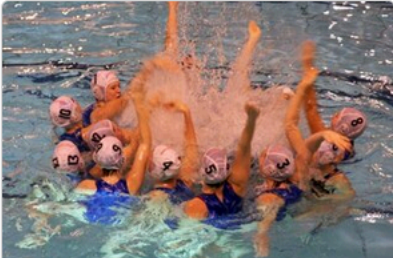
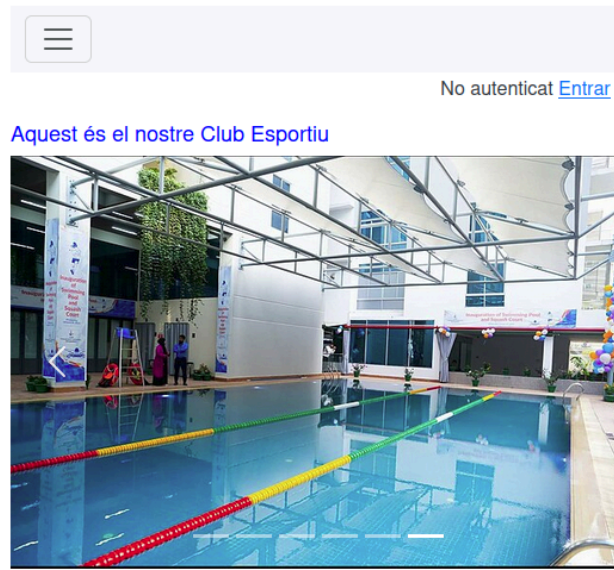
		
Duos Sincro	2x100m Papallona	4x100m Lliures
Natació Sincronitzada, modalitat duos	200 metres en modalitat papallona	400 metres crol
Grupal	Individual	Individual
24 agosto 2012	26 agosto 2012	25 agosto 2012
Classificacions de la prova Duos Sincro	Classificacions de la prova 2x100m Papallona	Classificacions de la prova 4x100m Lliures
		
4x100m Esquena	4x100m Estils	Combo
400 metres esquena	100 metres de cada estil	Natació Sincronitzada, modalitat combo
Individual	Individual	Grupal
27 agosto 2012	25 agosto 2012	25 agosto 2012
Classificacions de la prova 4x100m Esquena	Classificacions de la prova 4x100m Estils	Classificacions de la prova Combo

Figura 6. Exemple de l'ús de targetes amb Bootstrap, afegint imatges a les proves i al projecte.

Campionat de Natació



EI1027 - Disseny i Implementació de Sistemes d'Informació

Figura 7. Exemple d'índex dinàmic, que implementa un [carrusel d'imatges](#) (slider) i una [barra de navegació responsiva](#) que, en pantalles xicotetes, es contrau sota una icona de menú hamburguesa (les tres línies horitzontals) per optimitzar l'espai.

Campionat de Natació

[PÀGINA PRINCIPAL](#) [PROVES I CLASSIFICACIONS](#) [ÀREA PRIVADA](#)

No autenticat [ENTRAR](#)

Competicions

INDIVIDUAL GRUPAL TOTES



2x100m Papallona

200 metres en modalitat papallona

Individual

26 agosto 2012

[CLASSIFICACIONS DE LA PROVA 2X100M PAPALLONA](#)



4x100m Lliures

400 metres crol

Individual

25 agosto 2012

[CLASSIFICACIONS DE LA PROVA 4X100M LLIURES](#)



4x100m Esquena

400 metres esquena

Individual

27 agosto 2012

[CLASSIFICACIONS DE LA PROVA 4X100M ESQUENA](#)



Figura 8. Exemple de filtre aplicat al llistat de proves amb paràmetre opcional: la selecció del tipus en el grup de botons s'envia al controlador com a paràmetre opcional (prova/ListFiltre#Individual). El format és Sandstone de Bootswatch.

5. Enllaços d'interés

- Thymeleaf page layouts:
<https://ultraq.github.io/thymeleaf-layout-dialect/>
<https://www.thymeleaf.org/doc/articles/layouts.html>
- HttpSession:
<https://docs.oracle.com/javaee/6/api/javax/servlet/http/HttpSession.html>
- Acceso a atributos de sesión en Thymeleaf:
<https://docs.oracle.com/javaee/6/api/javax/servlet/http/HttpSession.html>
- Otras variables y objetos accesibles con Thymeleaf:
<https://www.thymeleaf.org/doc/tutorials/3.1/usingthymeleaf.html#appendix-a-expression-basic-objects>
- Jasypt:
<https://javadoc.io/doc/org.jasypt/jasypt/latest/index.html>
- Algunas soluciones para problemas típicos en las prácticas (accesible desde l'Aula Virtual)
<https://aulavirtual.uji.es/mod/url/view.php?id=5617729>